

Multiparty computation and secret sharing

2025

Examples

1 Introduction

Multiparty computation was once a mere theoretical topic: entertaining (theoretical) cryptographers but seemingly irrelevant (because deemed impractical) for real-world applications. As with so many attempts to predict the “practicality” of an idea, the reality turned out to be different: what once was seen as impractical is now a fundamental enabler for many privacy enhancing technologies that can be found “in the wild”.

The term multiparty computation encompasses different ideas about what and how multiple parties want to compute. Focusing on cryptographic use cases only, the aim of the computation could be to manipulate some data, but it could also be to manipulate some key material. It turns out that the latter use case is also relevant for embedded systems security, where leakage on keys can significantly weaken the actual security of systems.

1.1 Ideas and Concepts Covered in this Write-up

- Examples for multiparty computation
- Problems solvable by multiparty computation
- Techniques for multiparty computation: secret sharing and garbled circuits

2 Some Examples for Motivation

Classical cryptography delivers solutions for situations where a single party typically manipulates some data and then wishes to either securely store data, or, send this data securely to some other party (thus there might be multiple parties involved).

The difference to multiparty *computation* is in the latter word: rather than just storing or sending data between multiple parties, we also wish to do some computation on data (and this may include key material).

There two families of techniques for practical multi-party computation. One technique is based on constructing cryptosystems that are “homomorphic”, i.e. the encryption

function Enc has the property that $Enc(x_1 \cdot x_2 \cdots x_n) = Enc(x_1) \star Enc(x_2) \star \cdots \star Enc(x_n)$. Here the operators $\cdot$ and $\star$ refer to some arithmetic functionality. The idea is then that if there are different distrusting parties who each own an x_i , they can compute $Enc(x_1 \cdot x_2 \cdots x_n)$ without having to reveal their x_i values (by finding a function Enc and corresponding operators $\cdot$ and $\star$).

The other technique is called secret sharing. Here is the idea that there is a secret value x . Perhaps this value needs to be shared among multiple distrusting parties, or maybe one needs to demonstrate knowledge of this value without revealing it. The idea is then to represent this value via a tuple of random data, whereby a suitable recombination of the random data gives in fact x . We will largely focus on secret sharing techniques in this course.

A special case for multiparty computation is when only two parties wish to perform some joint computation. In this case the technique of garbled circuits is used in practice — it is neither based on homomorphisms nor on secret sharing.

2.1 Computing an average over secret data

A classical example, found in many introductory texts and tutorials, is that of multiple entities wishing to compute an average of some data, but nobody wishes to reveal the data that they have. For instance, multiple people would like to know if they earn above or below the average, but they are not willing to reveal their salary to anyone. At a first glance, one might conclude that if nobody is willing to trust anybody, then no computation can take place, but there is actually a solution to this problem, which is based on a technique called secret sharing.

Suppose that we have n different people, each with a secret salary s_i . What we wish to know is $\frac{1}{n} \sum_{i=1}^n s_i$. Thus strictly speaking, we only need to find a way that each party can compute the sum of s_i values. But we still have the problem of how to get the s_i values to all parties because they are meant to stay secret!

The perhaps “obvious” solution to this problem (at least for somebody who has had some introduction to cryptography) is to kind of “encrypt” s_i . The issue is that we need to find a way to encrypt s_i such that manipulation of the ciphertext leads to some useful manipulation over the plaintext. This can be done, but in this case here, there is a simpler solution.

Let’s stick with the idea of “encrypting” s_n though for a moment: the simplest encryption

scheme that all of us are familiar with is the one-time pad. It consists of taking a secret value (typically a single bit of information), which is “masked” by a secret key value (typically also a single bit of information): so if our salary s_n was a bit, then we could “encrypt” it via $c_i = s_i + k_1 + k_2 + \dots k_{n_1}$ (k_i are randomly chosen, this is already an extension of the one-time pad, where I allow multiple key bits to be added to the secret). We know that the one-time pad offers information-theoretic security: seeing c_i reveals nothing about s_i . In fact, the “roles” that the variables play in the one-time pad are somewhat interchangeable: neither c_i nor any of the k_i reveal anything about s_i .

Now let’s play a little bit more with this idea: suppose that I choose n values to represent each salary $s_i = s_i^1 + s_i^2 + \dots + s_i^n$. Evidently the sum over all s_i can be computed by summing over all s_i^j . Perhaps the next idea becomes now somewhat obvious: whilst nobody is willing to reveal s_i , it is safe to reveal all but one of the s_i^j to different parties:

$$\begin{array}{cccccc}
 P_1 & P_2 & \cdot & \cdot & \cdot & P_n \\
 s_1^1 & s_1^2 & \cdot & \cdot & \cdot & s_1^n \\
 s_2^1 & s_2^2 & \cdot & \cdot & \cdot & s_2^n \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 s_n^1 & s_n^2 & \cdot & \cdot & \cdot & s_n^n
 \end{array}$$

Now each party can compute part of the overall sum, without learning anything about the s_i : P_i can compute $\sum_{j=1}^n s_j^i$. With each partial sum, the overall sum can also be computed: each party simply passes on their sum to the other parties.

Perhaps a slightly more mathematical view here would be to just look at different ways in which the elements of the above matrix can be summed: we can do this row-wise or column-wise (and our solution sums them column-wise).

Our solution to this real world problem makes some implicit assumptions: we assume that all parties are playing nicely (i.e. they are not intending to inject corrupt data, they pass on correct result, they are online and participating, they are not colluding to reveal information about other parties).

I developed this example from the view-point of somewhat “encrypting” the data, inspired by the one-time pad. The core idea, that I would like you to take away, is that we can represent a secret value s by a suitable function of two or more numbers s^i , and assuming that we choose the s^i randomly and independently of s , then nothing can be learned about s from one or *allbutone/some* of the s_i .

2.2 Two-party communication via a garbled circuit

The case where two parties want to compute a function over their private inputs can be (relatively efficiently) solved by a protocol called garbled circuits. This idea was initially explained by Andrew Yao in a talk, and thus we find this technique often referred to as “Yao Circuit” in the literature. Suppose that the two parties want to know who has a higher salary, without needing to reveal their salaries. In the following description, we call one party the “garbler”, and the other party the evaluator. The function that they wish to evaluate is represented as a Boolean circuit (i.e. it can be represented by Boolean functions like AND, XOR, etc.)

The protocol proceeds in several steps, one of which requires the use of what is called an “oblivious transfer” protocol. For the sake of brevity, I will omit explaining how this exactly works for now.

1. The garbler “encrypts the circuit”, and also their input.
2. The garbler sends the encrypted circuit plus their encrypted input to the evaluator.
3. The evaluator and garbler engage in an oblivious transfer that enables the evaluator to get the encryption of his input without the garbler learning anything about the input.
4. The evaluator evaluates the encrypted circuit using both encrypted inputs to get the encrypted result.
5. The evaluator and garbler communicate to reveal the output of the function.

2.3 Manipulating a secret value in the presence of information leakage

Another, perhaps slightly left-field, example for the utility of secret sharing relates to the situation where an adversary can observe “something” that relates to the manipulation (e.g. loading, storing, calculating) of a secret value. This situation can be often found in the context of embedded systems security, where an additional challenge is that the device storing and manipulating the secret, is not in a secure environment. Thus physical access to the device is “easy”.

Let us assume that an adversary can observe the power consumption of values being loaded: i.e. if s is fetched from memory, the adversary can observe $f(s)$ (we assume for simplicity now that $f()$ is known to them, and $f()$ does not contain any randomness). If

$f()$ is one-to-one, then the adversary can deduce s from observing $f(s)$. If $f()$ is injective, then the adversary cannot learn s precisely, but they can learn the pre-image of $f(s)$.

Example. Suppose that a device leaks the Hamming weight (HW) of data values, and that a data value can have no more than 8 bits. So if the value 3 is manipulated, the device leaks $HW(3) = 2$. If an adversary thus sees the leakage 2, they know that the intermediate value was either $\{3, 5, 6, 9, 10, 12, 17, \dots\}$.

The technique of secret sharing s from before is helpful in this context. Assume that rather than storing s , the device stores only shares s^i . Thus upon loading the request to fetch s , the device fetches the shares s^i instead. Because the s^i were drawn randomly and independently from s , they contain no information about s . Thus any function $f(s^i)$ gives no information about s . Consequently, observing just one (or all but one) $f(s^i)$ gives an adversary no whatsoever advantage to recover s . Only if the adversary can observe all $f(s^i)$ can they learn something about s .

Example. Suppose that instead of manipulating the value 3, we manipulate 75 and 72. Why? Check out the binary representations and observe that $3 = 75 \oplus 72$ ($\oplus$ equals addition modulo 2). Now the observable leakages would be 2 and 4. Although the adversary knows that we secret shared the value $s = s_1 \oplus s_2$, if they only get one of the two observations, they learn nothing about s .

You may wonder why the adversary, in the case of leakage, should not be able to see all $f(s^i)$: in practice we would expect just that. If they see the leakage of both shares, then they can enumerate all pre-images for both leakages, and then try out all combinations of numbers in the pre-image sets. This leads potentially to a large number of combinations to try, but it is practically feasible. Thus, clearly I am not telling you the full story here!